

NP Theory Homework

In this assignment you will use the provided code template to implement solutions to each given problem. The solutions you submit must follow the guidelines provided in this document as well as any given in code.

Restrictions

- Template code is provided and must be used.
 - ***Note: The templates are refreshed each semester. Be sure to use the current version.***
- Your code must be compatible with **Python 3.10**.
- Do not add imports for any libraries beyond the Python standard library.
- Do not modify the function name or definition.

Testing

For each problem, one or more base test cases are provided. A properly implemented solution will pass any provided base test cases, but their trivial nature is not intended to ensure the overall correctness of your algorithm.

These are the same test cases that will be used to provide assurance that your submitted solutions run using the autograder. We do not release any other test cases.

In problem instances where more than one solution is optimal, *any* optimal solution will be considered correct, assuming that is the solution your algorithm produces.

The test cases can - and should! - be expanded while developing your solutions.

You can run your test cases by issuing the `python3 -m unittest` command from the directory containing your solution files.

Submission

Submit only - and all of - your solution files (i.e., `cs6515_[problem_name].py`) to the Gradescope assignment on or before the posted due date. Do not submit a zip file or any other files except those matching the naming convention defined.

You may submit your solutions as many times as desired and the most recent submission is what will be graded. Execution of your code may take up to 60 minutes. If not completed before the due date, your previous submission will be used.

Faster and correct solutions are worth more credit.

After your submission completes execution, you will see a score of - / 20 listed until grading is completed.

Double SAT

The Double SAT problem takes as an input, a boolean function in conjunctive normal form, and outputs two **distinct** assignments of the variables such that all clauses evaluate to true, or reports No Solution if no such pair of assignments exists.

There may be more than two satisfying assignments, but Double SAT only returns two.

Examples:

- $(a \vee b)$
 - $a, !b$
 - $!a, b$
- $(a \vee b) \wedge (!a \vee b)$
 - a, b
 - $!a, b$
- (a)
 - No Solution

Unfortunately due to budget concerns (Joves' hourly pay) we can only afford to verify against a reduction from SAT. So while we know that any NP-Complete problem can reduce to any NP-Complete problem, for this programming assignment you are tasked with reducing from SAT $\rightarrow$ Double SAT.

Show that Double SAT is NP-Complete **by reducing from SAT**.

You will need to implement the following functions:

- `verify_solution`
- `input_transformation`
- `output_transformation`

You may assume the following (to simplify your code):

- The input to SAT are all lower case letters.
- The input to Double SAT are both lower and upper case letters.
- A variable can be a combination of letters such as `validvariablename`

Please read `test_double_sat.py` to see how we will be testing your functions.

Provide your code for implementing this NP-Complete proof in `cs6515_double_sat.py`